SiPAM: Silicon Photonic Accelerated Memory Pooling Architecture

Zhenguo Wu, Yuyang Wang, and Keren Bergman, *Columbia University, New York, NY, 10027, USA*

Training and inference of large-scale machine learning models demand significant processing power, memory capacity, and memory bandwidth. Current compute units are equipped with a fixed amount of high-bandwidth memory modules placed near the compute die which limits the system's flexibility to adapt to different compute and memory demands. In this work, we present SiPAM, a silicon photonic accelerated memory pooling architecture that integrates dense wavelength-division multiplexing–based photonic input–outputs directly along the compute die's shoreline. This design creates a unified high-bandwidth communication domain that decouples memory and network scaling from packaging constraints and enables demand-driven dynamic resource allocation. We develop a quantitative optimization methodology to allocate compute and memory resources efficiently, with up to 3.5× faster iteration time and 4.5× reduction in energy consumption compared to state-of-the-art compute systems.

The growing adoption of machine learning (ML) applications has placed significant demands on computation, memory capacity, and memory bandwidth in modern compute systems. In particular, the rapid scaling of large language models (LLMs) has introduced two fundamental challenges for scaling system efficiency.

First, the total memory required to store model parameters, activations, and optimizer states has grown to tens of terabytes, which well exceeds the capacity of locally attached high-bandwidth memory (HBM) on a single compute unit (CU) such as GPUs or TPUs. As a result, model states and datasets are distributed across a large number of CUs during training and inference, which introduces significant inter-CU communication overhead and reduces overall system efficiency.

Second, memory bandwidth has scaled at a slower rate than compute throughput across GPU generations. This mismatch is particularly limiting for workloads with low arithmetic intensity, where performance is bounded by memory bandwidth rather than compute throughput. In practice, arithmetic intensity varies not only across different workloads but also within different execution stages of a workload. For example, inference generally has lower arithmetic intensity than training, and within inference, the decode stage is more memory-bandwidth sensitive than the prefill stage.

These limitations stem from the packaging constraints that require HBM stacks to be integrated near the compute die which has a fixed shoreline width. As a result, the compute systems are provisioned with a fixed ratio of compute throughput to memory bandwidth, resulting in a static hardware compute intensity that cannot adapt to the varying arithmetic intensity demands of different ML workloads.

To improve memory capacity and flexibility, memory pooling has been proposed, where CUs have access to a shared pool of remote memory units (MUs). In current architectures, local HBMs serve as the high-bandwidth supplier, while remote MUs (e.g., double data rate [DDR] or graphics double data rate [GDDR]) connected over electrical fabrics are used for capacity expansion.[1] This memory hierarchy arises from the physical integration constraints of HBM and the latency and bandwidth constraints of remote MUs.

To address these challenges, we propose a photonic solution that integrates high-bandwidth, dense wavelength-division multiplexing (DWDM)–based silicon photonic (SiP) input–outputs (I/Os) directly along

© 2026 The Authors. This work is licensed under a Creative Commons Attribution 4.0 License. For more information, see https://creativecommons.org/licenses/by/4.0/.
Digital Object Identifier 10.1109/MM.2026.3670009
Date of publication 5 March 2026; date of current version 18 August 2026.

the shoreline of the CUs with up to 240 Tbps per-CU injection bandwidth, fully replacing conventional electrical I/Os that are constrained in bandwidth and reach. Each SiP I/O can be flexibly allocated for memory access or network communication through an optically switched interconnect fabric. To dynamically allocate compute power, memory capacity, and I/O bandwidth based on parallelism strategies and resource constraints, we also propose a roofline-based optimization methodology. Extending our prior work in Wu and Bergman,[2] we conduct a power and energy analysis to examine the performance-energy tradeoffs of SiPAM. Evaluations show that our solution scales effectively with model size, improves memory utilization, and projects up to 3.5× faster iteration time and up to 4.5× reduction in energy consumption compared to state-of-the-art compute systems.

BACKGROUND AND RELATED WORKS

Artificial Intelligence Compute Systems and the Memory Wall

To accommodate larger ML models, systems employ a combination of parallelism strategies which result in different memory and communication requirements. As a result, memory provisioning directly influences both the choice and efficiency of parallelism.[3]

A fundamental constraint on memory integration is the shoreline width of the compute die. As shown in Figure 1(a), current architectures integrate HBM stacks along the die periphery using short-reach electrical interfaces. Due to their high pin counts and constraints on signal integrity and energy consumption, these interfaces restrict HBM placement to locations close to the compute die. As a result, the number of HBM stacks is limited by the package size and die perimeter, restricting memory capacity scaling and its ability to adapt to changing memory demands.

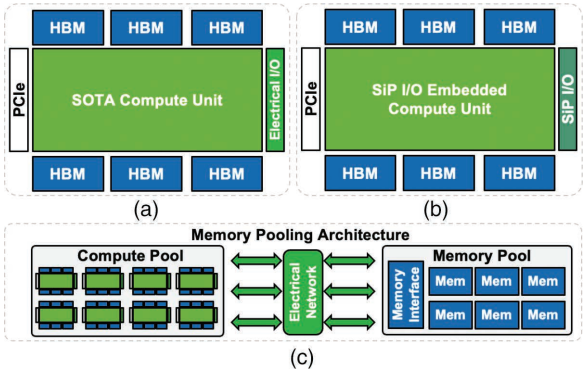


FIGURE 1. (a) Current compute architecture with on-package HBM stacks and high-speed electrical I/Os for networking. (b) CU with SiP I/Os replacing electrical interfaces. (c) Conventional memory pooling architecture connecting compute pools to memory pools using an electrically switched network.

Memory Pooling

Memory pooling has emerged as a promising architectural approach to overcome the limitations of the memory bottleneck. As shown in Figure 1(c), it allows CUs to access a larger and more flexible pool of memory. In traditional architectures, remote MUs are connected via peripheral component interconnect express (PCIe) or compute express link-based I/Os and electrically switched networks.[1] These systems rely on local HBMs as high-bandwidth memory supplier and use slower remote MUs (e.g., DDR, GDDR) as capacity expanders for offloading.[3] This hierarchical design introduces added memory latency due to protocol overhead and physical distance. Recent work[1] shows that this latency can be effectively masked under full system load when sufficient bandwidth is provisioned.

Copackaged Silicon Photonic I/Os

To meet the bandwidth demands of memory pooling, silicon photonic I/Os copackaged with compute chips have emerged as a promising solution. Recent industry developments in optical I/O integration, such as copackaged optics, have demonstrated the feasibility of integrating optical transceivers directly with switch application-specified integrated circuits (ASICs), suggesting a viable path for bringing optical bandwidth directly to the compute die. Moreover, recent studies have shown frequency-comb-driven DWDM transceivers to achieve up to 4iTbps/mm bandwidth densities and sub-pJ/bit energy efficiency.[4] Architecturally, prior works have proposed replacing conventional electrical I/Os on CUs with SiP I/Os to enable higher escape network bandwidth,[5] as shown in Figure 1(b). This increased bandwidth, combined with the use of optical circuit switching, has been shown to improve the efficiency of collective communication in ML workloads. In addition, prior work on optically connected multistack HBM has shown the copackaged 3-D integration of electrical interface chiplets (EICs) and photonic integrated circuits (PICs),[6] highlighting the feasibility of using HBM modules as remote MUs to improve system efficiency for training and inference workloads.[6]

SYSTEM ARCHITECTURE

In this section, we present the SiPAM compute and network architecture. A list of mathematical symbols used are summarized in Table 1.

SiPAM Compute Architecture

To address the capacity and bandwidth limitations of HBM, we propose to consider the compute die's available shoreline width as a critical design resource. Building on prior works,[4,6] we propose a multichip module (MCM) approach that embeds SiP I/Os composed of 3-D-integrated EIC and PIC directly along the compute die's edges. Figure 2(a) illustrates our proposed SiPAM architecture, where the compute die's periphery (excluding the PCIe interface for CPU connectivity) is populated with SiP I/Os. Figure 2(b) further illustrates the integration of EIC and PIC through flip-chip bonding. Unlike prior designs that replace each HBM with one SiP I/O,[6] our approach considers both the total die shoreline width (W_d) and the edge dimension of SiP I/O (W_{io}) to determine the number of integratable SiP I/Os as $N_{io} = \lfloor W_d/W_{io} \rfloor$ in our analysis. Recent work[4] has pushed the limits of shoreline bandwidth density by demonstrating an optical frequency comb–driven SiP I/O chiplet comprising 16 links, each operating across 64 wavelength channels modulated at 16 Gbps per channel. This design supports up to $16 \times 64\lambda \times 16$ Gbps ≈ 16 Tbps $= 2$ TBps of bandwidth within a compact edge footprint of $W_{io} = 8.1$mm, achieving a bandwidth density of 2 Tbps/mm. With 32 Gbps/channel, this bandwidth density can be doubled to 4 Tbps/mm.

For the memory pool, we propose to have multiple MUs integrated on a single chiplet to match the bandwidth of a single SiP I/O, similar to Ou et al.[6] We use HBMs as an example of MUs, but our analysis could be extended to other memory types. The number of remote MUs integratable per I/O chiplet is determined by the ratio of SiP I/O bandwidth (B_{io}) to per MU bandwidth (B_m), given by $N_m = \lfloor B_{io}/B_m \rfloor$.

SiPAM Network Architecture

To address the rigid compute-to-memory bandwidth ratio and build a unified communication domain, each SiP I/O can be flexibly allocated for memory access or network communication through an optical circuit switched (OCS) fabric. SiPAM adopts the OCSBCube physical topology, similar to Wu et al.[5] An example SiPAM ($n = 2, k = 8$) is shown in Figure 2(c), totaling 64 CUs, comparable in scale to the NVIDIA NVL72 rack architecture[7] which includes 18 compute trays with four GPUs each. SiPAM repurposes a subset of these compute trays as memory trays containing MUs connected via OCSes through SiP I/Os [Figure 2(d)]. This design adopts the intra-rack resource disaggregation model for a bounded increase in memory latency. The packet-switch-less design eliminates electrical-to-optical conversions and further reduces in-network delays for low-latency remote memory access.

OPTIMIZATION METHODOLOGY

To dynamically allocate compute power, memory bandwidth, and memory capacity based on workload need, we propose a roofline-model-based optimization methodology. System performance under this model is determined by the lower of two factors: the hardware's peak floating-point operations per seconds (FLOPs) performance or the product of memory bandwidth and workload arithmetic intensity. Workloads with arithmetic intensity lower than the compute intensity are memory-bound, as performance is constrained by the memory bandwidth, whereas those with higher arithmetic intensity are compute-bound, limited by the peak compute FLOPs. The optimal operating point occurs when arithmetic intensity matches compute intensity, ensuring that both compute and memory bandwidth are most efficiently used. Since memory bandwidth in the SiPAM architecture is a reconfigurable parameter, our approach involves designing the hardware such that the compute intensity aligns with the arithmetic intensity to achieve optimal efficiency.

TABLE 1. List of mathematical symbols and their descriptions.

Notation	Description
F_{peak}	Peak compute FLOPs/s per CU
I_w	Arithmetic intensity of a workload
N_{cu}	Number of CUs
W_d	Available compute die shoreline width
W_{io}	Edge width per SiP I/O
N_{io}	Number of integratable SiP I/Os per CU
$N_{io,m}$	Number of SiP I/Os allocated for memory
$N_{io,n}$	Number of SiP I/Os allocated for network
N_m	Number of connected MUs per CU
B_{io}	Bandwidth per SiP I/O
B_m	Bandwidth per MU
B_{req}	Required memory bandwidth per CU
B_{cu}	Total memory bandwidth per CU
C_w	Capacity needed to fit a workload
C_m	Capacity of a single MU
C_{cu}	Total memory capacity of a CU

First, given the peak compute FLOPs/s, F_{peak}, of the CU and the arithmetic intensity, I_w, of the workload, the minimum required memory bandwidth can be calculated as $B_{req} = F_{peak}/I_w$ which corresponds to the ridge point on the roofline plot. We assume that this ridge point stays invariant with increasing number of CUs, as both compute power and memory bandwidth increase proportionally with additional CUs. Next, each MU in the memory pool provides a fixed bandwidth of B_m, allowing us to determine the total number of required MUs: $N_m = \lceil B_{req}/B_m \rceil$. This yields a per-CU memory bandwidth of $B_{cu} = N_m B_m$ and a per-CU memory capacity of $C_{cu} = N_m C_m$. From this, we can determine the number of SiP I/Os allocated for the memory pool: $N_{io,m} = \lceil B_m/B_{io} \rceil$, where B_{io} is the bandwidth per SiP I/O. Any remaining I/Os are allocated for inter-CU network connections, with a minimum guarantee of one link dedicated to each connection in the SiPAM topology.

Given the available memory capacity per CU, the *estimated* number of CUs required to fit the workload is $\lceil \tilde{N}_{cu} = C_w/C_{cu} \rceil$, where C_w is the estimated size of the workload. Given this estimated lower bound $\tilde{N}_{cu}$ and the total number of available CUs N_{cu} as the upper bound, we apply a binary search algorithm to determine the minimum number of CUs needed to fit the model. At each step, we exhaustively search through all feasible parallelization strategies, selecting the strategy that results in the lowest iteration time if the model fits under the current network size and per-CU memory capacity constraint. This algorithm is outlined in Algorithm. 1.

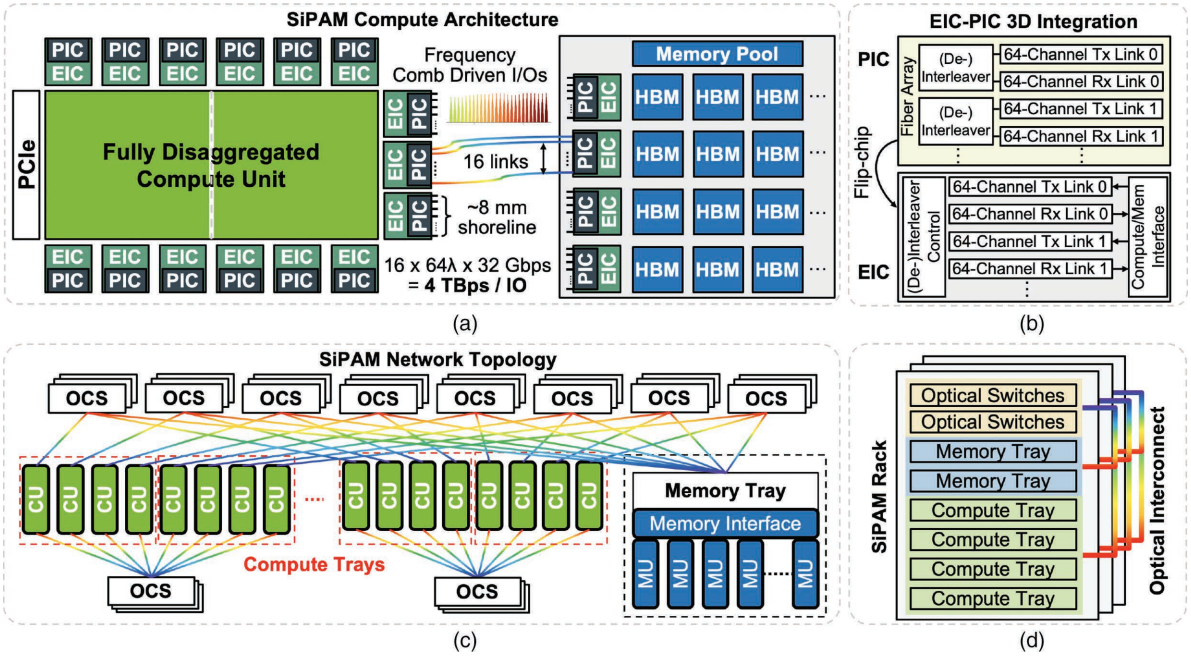


FIGURE 2. Schematic of (a) an example of the SiPAM compute architecture with frequency-comb-driven SiP I/Os embedded along the full periphery of a dual-die compute unit, (b) 3-D integration of the EIC and the PIC for the SiP I/Os,[4] (c) the SiPAM network architecture which realizes a multidimensional all-to-all topology while adopting the intra-rack memory pooling model, and d) the SiPAM intra-rack memory pooling structure for bounded latency.

TABLE 2. Specifications of GPUs and HBMs.[8,9,10,11]

			HBM		
GPU	**FP16 (TFLOPs)**	**Power (W)**	**Capacity (GB)**	**BW (GBps)**	**BW Density * (GBps/mm)**
A100	312	400	40	1,555	25.6
H100	1000	700	80	3000	50.0
B100	3500	1000**	192	8000	83.3

* Estimated using approximate HBM die dimensions. ** Estimated value.

Algorithm 1: BINARYSEARCH for Feasible # of CU

Require : $\tilde{N}_{cu}$, N_{cu}, C_{cu}
Ensure : Min CU count and corresponding parallelism config (if any)

$L \leftarrow \tilde{N}_{cu}$, $R \leftarrow N_{cu}$, $optim_config \leftarrow \emptyset$;
while $L \leq R$ **do**
  $m \leftarrow \lfloor (L+R)/2 \rfloor$;
  configs $\leftarrow$ `AllPossibleConfigs`(m) ;
  valid $\leftarrow$ `ExhaustiveSearch`(configs, C_{cu}) ;
  if valid $\neq \emptyset$ **then**
    optim_config $\leftarrow$ $\text{argmin}_{c \in \text{valid}}$ `IterationTime` (config) ;
    $R \leftarrow m-1$; *// Try fewer CUs*
  else
    $L \leftarrow m+1$; *// Need more CUs*
// No valid configuration found if optim_config == ∅
return (L, optim_config) ;

Based on the result of the parallelization search algorithm, the workload's arithmetic intensity and bandwidth requirements may deviate from initial estimates. In such cases, we revisit and adjust prior configuration steps, including memory configurations and estimated CU count, to reflect the updated requirements. This iterative refinement continues until convergence to a stable configuration or a predefined iteration limit, T_{max}, is reached, which we consider the optimal solution. The final solution N^{*}_{cu} denotes the number of CUs required to host the workload under the optimal parallelization configuration. The full iterative algorithm is outlined in Algorithm. 2.

Algorithm 2: OPTIMALCONFIG for Resource Allocation

Require : C_w, I_w, F_{peak}, B_m, B_{io} N_{cu}, T_{max}
Ensure : Stable configuration of CU count and parallelism (if found)

for $t \leftarrow 1$ *to* T_{max} **do**
  // Determine estimated # of CUs and capacity per CU
  $(\tilde{N}_{cu}, C_{cu}) \leftarrow$ `Optimize` $(C_w, I_w, F_{peak}, B_m, B_{io})$;
  $(N^{*}_{cu}, \text{optim_config}) \leftarrow$ `BinarySearch` $(\tilde{N}_{cu}, N_{cu}, C_{cu})$;
  if $N^{*}_{cu} \neq$ `None` ***and*** optim_config *unchanged* **then**
    // Converged to stable configuration
    return $(N^{*}_{cu}, \text{optim_config})$;
  else *// Update workload parameters*
    $(I_w, C_w) \leftarrow$ `Update`(N^{*}_{cu}, *optim_config*) ;
// Best config after T_{max} iterations
return $(N^{*}_{cu}, \text{optim_config})$;

SYSTEM EVALUATIONS

In this section, we evaluate our proposed SiPAM architecture across diverse hardware configurations and LLM workloads. Specifically, we examine how system performance scales with model size, HBM generation, and total available compute resource. We compare SiPAM against state-of-the-art systems, highlighting improvements in iteration times and energy efficiency.

Evaluation Method

We use Calculon,[3] an open source, analytical DL performance model, to evaluate the performance of the SiPAM architecture on Transformer-based LLMs with sizes ranging from 126 million to 1 trillion parameters. Total model size includes memory for weights, activations, weight gradients, activation gradients, and optimizer states. We assume a baseline batch size and sequence length of 2048 unless otherwise specified.

For compute hardware, we model three GPU generations (i.e., A100, H100, and B100) with peak throughput as summarized in Table 2. For memory hardware, we model three HBM generations: HBM2 (8 GB at 307 GBps/stack), HBM2e (16 GB at 600 GBps/stack), and HBM3e (24 GB at 1000 GBps/stack), which correspond to the three GPU generations, respectively. These configurations serve as baselines for performance comparison against SiPAM. In the A100 and H100 systems, GPUs are organized into NVSwitch/NVLink clusters of 8, with intra-cluster bandwidths of 600 GBps and 900 GBps. For the B100 system, the NVLink domain has 72 GPUs[10] at 1800 GBps per GPU. We assume that inter-cluster communication is supported by InfiniBand or Ethernet at 800 Gbps per GPU.

Workload Analysis

We begin by analyzing SiPAM's performance across a range of LLM workloads. As a baseline, we model the NVIDIA B100 compute architecture as outlined in Table 2. For SiPAM, GPUs has the same TFLOPs/s, while each MU provides 24 GB of capacity and 1000-GBps memory bandwidth.

From Figure 3(a) and (b), we observe that across different workloads, SiPAM's compute intensity closely tracks each workload's arithmetic intensity for both training and inference, and the slight differences are due to rounding operations during optimization. In contrast, the baseline compute intensity remains constant across different workloads since its compute-to-memory bandwidth ratio is fixed. Figure 3(c) and (d) shows the iteration time for training and inference normalized to SiPAM's performance for each

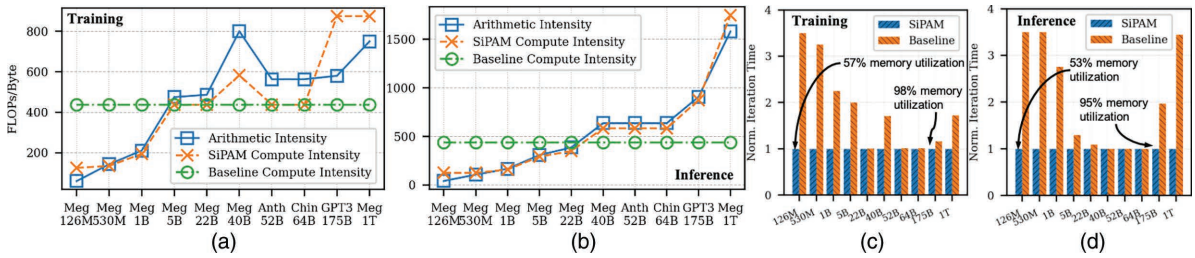


FIGURE 3. Arithmetic and compute intensity for (a) training and (b) inference of various Transformer-based workloads. Normalized iteration completion time for (c) training and (d) inference. We label percentage memory utilization for two representative cases: Memory-bound workloads are provisioned with additional capacity to satisfy bandwidth demands, whereas compute-bound workloads are allocated only the capacity required to fit the model.

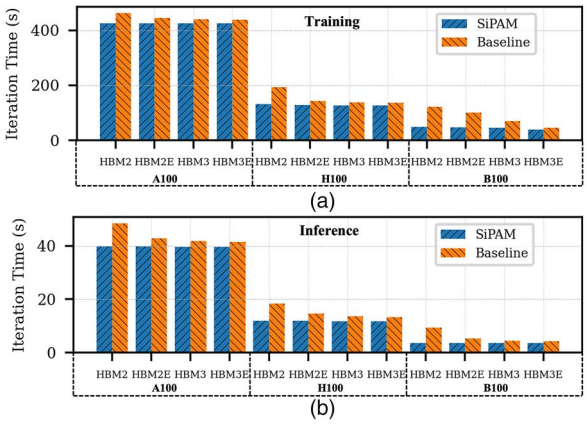


FIGURE 4. Iteration time across different combinations of compute and memory technology pairs for (a) training and (b) inference of GPT3-175B.

workload. We observe that SiPAM consistently outperforms the baseline by optimally allocating the needed bandwidth resources based on each workload. This results in up to 3.5× reduction in iteration time for training and inference. We notice that our proposed methodology is particularly effective for workloads whose arithmetic intensity differs substantially from the baseline compute intensity, as it can provide optimal resource allocation. For workloads with arithmetic intensity matching the compute intensity of the baseline, we observe minimal gains from SiPAM. We note that the significant increase in memory bandwidth comes at the cost of higher memory capacity provisioning. This overprovisioning is by design which allows bandwidth scaling beyond package constraints. We discuss the associated power and performance tradeoffs in the next section and show that, when performance is normalized by total system power, SiPAM continues to outperform the baseline architecture.

Compute and Memory Technology Scaling

We next evaluate how SiPAM scales across successive generations of compute and memory technologies. For compute, we model the NVIDIA A100, H100, and B100 GPUs. While each GPU generation is typically paired with a corresponding HBM generation, we also explore cross-generation pairings to assess the performance impact of decoupling compute and memory technologies. This analysis highlights SiPAM's flexibility in supporting heterogeneous hardware configurations. We fix the network size to 128 GPUs and use GPT-3 with 175 billion parameters (GPT-3 175B) as a representative large-scale workload to evaluate training performance.

Figure 4 shows the iteration time for various combinations of compute and memory technology pairs. We first observe that, for each GPU generation, performance improves as the memory generation advances from HBM2 to HBM3e in the baseline case, reflecting the benefits of increased memory bandwidth. Similarly, newer GPU generations outperform earlier ones due to higher compute throughput. Notably, SiPAM consistently outperforms the baseline as compute capability increases. This is because SiPAM can provision memory bandwidth that matches the compute demands through optimized SiP I/O bandwidth allocation. Within each GPU generation, SiPAM's performance remains relatively stable across HBM generations since its effective memory bandwidth scales with SiP I/O bandwidth rather than being directly limited by the physical HBM bandwidth, as is the case in baseline systems. We again observe that SiPAM achieves greater relative performance improvement during inference compared to training as inference is more memory bound and benefits more from increased memory bandwidth.

Performance Under Limited Compute Resource

We next evaluate SiPAM's performance under constrained compute resources. Specifically, we study how memory utilization and iteration time are affected as the number of available GPUs in the cluster varies. We consider two representative workloads for training: GPT-3 175B and Megatron-1T. For GPT-3 175B, we vary the cluster size from 16 to 128 GPUs to reflect smaller-scale deployments. For Megatron-1T, we scale the cluster size from 128 to 1024 GPUs.

In the baseline cases shown in Figure 5(a) and (b), red cells indicate configurations where no feasible parallelization strategy can accommodate the model given the available number of GPUs. These infeasible configurations become more frequent with smaller cluster sizes and with older GPU generations due to limited memory capacity. Among the feasible cases, darker shades represent higher iteration time. We observe that, for a fixed GPU-HBM combination, iteration time decreases as the number of available GPUs increases due to increased aggregate compute FLOPs. Similarly, for a fixed number of GPUs, newer GPU generations yield lower iteration time due to higher per-device compute FLOPs. From the in-cell texts that show memory utilization, we observe that, in the baseline case, memory usage remains nearly constant as cluster size increases. While a larger cluster size typically enables higher degree of parallelism and reduced memory requirements per GPU, the exhaustive search algorithm selects parallelization strategies that maintain similar memory usage but achieve lower iteration time to improve parallelism efficiency.

In contrast, SiPAM [Figure 5(c) and (d)] consistently enables feasible configurations across a wider range of cluster sizes and compute generations. Notably, SiPAM supports configurations that are entirely infeasible in the baseline case, such as training GPT-3 175B on 16 A100 GPUs and Megatron-1T on 256 A100 GPUs. Compared to the baseline case, which has a fixed memory capacity per configuration, SiPAM allows memory capacity to vary across configurations based on workload and system requirements. In general, memory provisioning decreases as cluster size increases. For example, on A100 GPUs with HBM2, the amount of provisioned memory capacity decreases from 176 GB to 56 GB for GPT-3 175B as the cluster size increases from 16 to 128, and from 144 GB to 56 GB for Megatron-1T as the cluster size increases from 128 to 1024. In some cases, SiPAM provisions more memory than the baseline, such as the 64 H100 GPU configuration for GPT-3 175B. This occurs because the optimization algorithm prioritizes allocating the required memory bandwidth to improve compute efficiency, which in this case corresponds to higher memory capacity. In other cases, such as the 128 H100 GPU configuration for GPT3-175B, SiPAM provisions less memory than the baseline, as it can meet bandwidth requirements with fewer MUs, achieving equivalent performance with less resource usage. For iteration time comparison, SiPAM consistently outperforms the baseline, as reflected by the lighter cell colors in the heatmap under feasible configurations. These results highlight SiPAM's ability to improve compute utilization by 1) providing sufficient memory bandwidth to sustain high compute efficiency, and 2) providing sufficient memory capacity to expand the parallelization design space to increase the likelihood of accommodating larger models and identifying more optimized configurations.

Power and Energy Analysis

To compare SiPAM with the baseline architecture in power and energy consumption, we first estimate power consumption by considering the contributions of all compute and networking components in a rack. The total rack power is obtained by multiplying the number of each component by its respective power consumption. Other components such as cooling

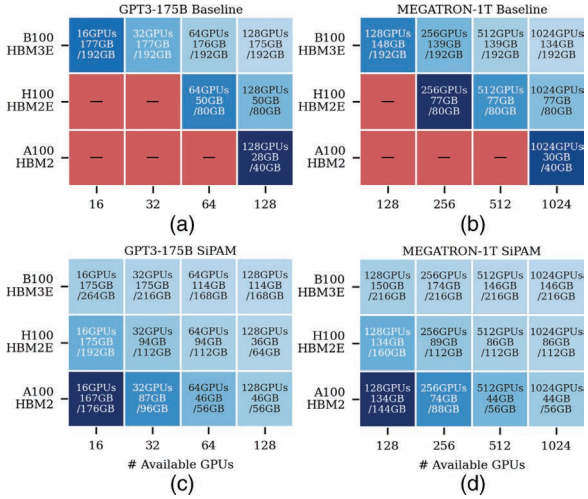


FIGURE 5. Heatmaps of iteration time for GPT-3 175B and Megatron-1T under varying cluster sizes and compute generations. (a) and (b) show baseline results; (c) and (d) show SiPAM results. Darker colors indicate higher iteration time. Red cells indicate no feasible parallelization strategy could fit the model. Text in cells show the number of actual GPUs used and memory utilization.

and facility infrastructure are not considered for this analysis.

In the baseline case, each rack contains 72 GPUs where each GPU is equipped with local HBMs and 18 electrical links (e.g., NVLinks) connecting to 18 in-rack EPSes (e.g., NVSwitches). We assume 600 GBps, 900 GBps, and 1800 GBps total bandwidth for each A100, H100, and B100 GPU, respectively. In the SiPAM configuration, each rack contains 64 GPUs, while the remaining trays are repurposed as memory trays to form an intra-rack memory pool. We assume that each rack is provisioned with sufficient MUs to fully utilize all SiP I/Os available to the 64 GPUs. The GPUs are arranged in a BCube physical topology with multiple planes of radix-8 OCSes. We assumed 18 planes to match the baseline case. The total bandwidth per GPU is assumed to be the product between available die perimeter length and SiP I/O shoreline bandwidth density (2 Tbps/mm).

For GPUs, we assume they consume 400 W,[8] 700 W[9] and 1000 W[10] for A100, H100, and B100, respectively. For HBMs, we assume they consume 10 W, 19 W and 32 W per stack for HBM2, HBM2e, and HBM3e following the roadmap outlined in https://www.aionet.tech/KAIST_HBM_Roadmap.pdf.[11] We then remove HBM power from total GPU power to avoid double counting. For links, we model the transceiver powers by multiplying the power density (in joules/bit) by the operating link bandwidth. We assume 1.3 pJ/bit[12] for the electrical links used in the baseline case. Optical links in the SiPAM architecture are modeled after the DWDM link design presented in Wang et al.,[4] which demonstrates an energy consumption of 432.4 fJ/bit, including the comb source, EIC for transmitter drivers and receiver transimpedance amplifiers, as well as thermal control power for modulators, filters, and interleavers integrated on the PIC. For electrical switches, power is scaled linearly from a 275 W, 72-port switch ASIC[5] to match the target port count and per-port bandwidth. For OCSes, we assume the commercial Calient 320-port OCS[5] with 45 W which translates into 0.15 W per port.

Figure 6 presents the power breakdown across GPU generations for both the baseline and SiPAM configurations. For A100- and H100-based systems, SiPAM exhibits higher total power consumption despite its reduced GPU count per rack, primarily due to the increased HBM aggregation within the memory pool. In the baseline case, power from the electrical link and switches increases significantly with bandwidth,

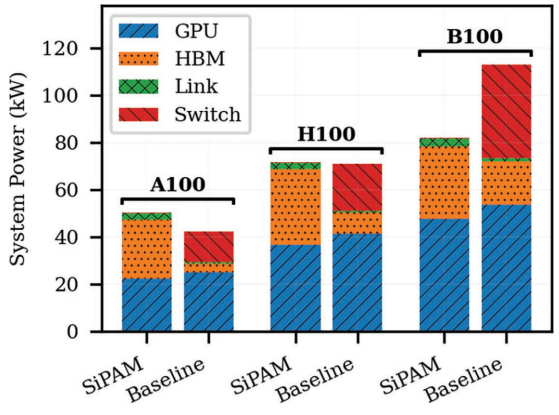


FIGURE 6. Power consumption breakdown of a rack-scale compute system across GPU–HBM generations, comparing the baseline configuration (72 GPUs per rack) and the SiPAM architecture (64 GPUs and a memory pool per rack).

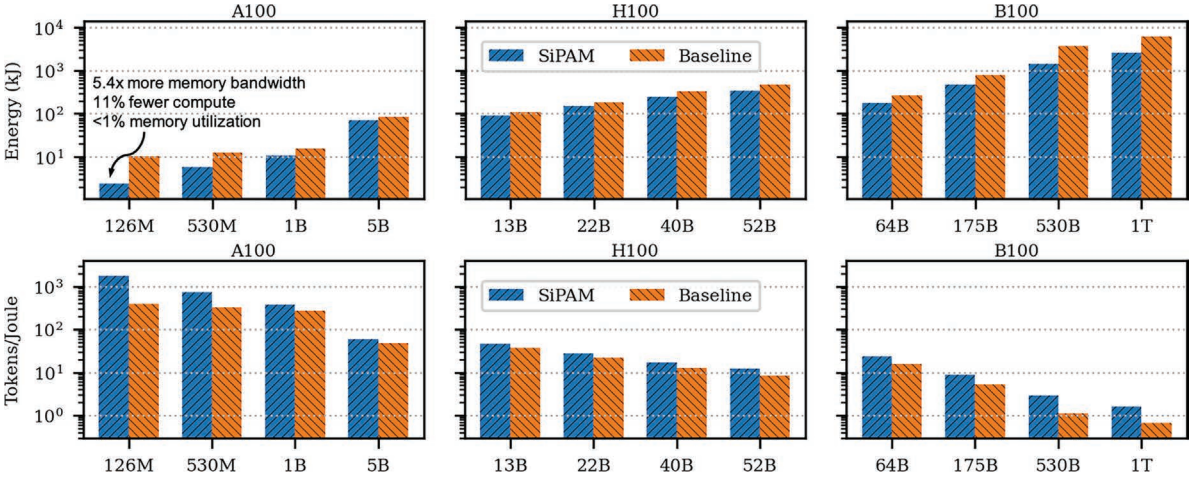


FIGURE 7. Comparison of inference energy consumption (top) and tokens per joule (bottom) across different workloads for SiPAM and baseline configurations.

whereas SiPAM's photonic interconnect scales more gradually due to the energy efficiency of the DWDM architecture. This results in the baseline's power to increase more than that of SiPAM in the B100-based system. Since OCS ports are bandwidth-independent, their power contribution remains negligible compared to other components.

To examine performance–energy tradeoffs, we evaluate SiPAM across a range of LLM inference workloads. Figure 7 shows energy consumption (top) and normalized throughput in tokens per joule (bottom) for both the baseline and SiPAM. We observe that SiPAM consistently achieves lower energy consumption with up to 4.5× reduction. As shown in the A100 energy plot, SiPAM provisions additional memory capacity to meet bandwidth requirements, which leads to higher memory power consumption and lower memory utilization. However, by operating at the optimal roofline point, SiPAM uses fewer compute resources and achieves better iteration time which results in lower overall energy consumption. In summary, although memory pooling introduces additional power overhead, its higher memory bandwidth improves inference iteration time and results in a more favorable performance–energy tradeoff than the baseline system.

CONCLUSION

In this work, we propose SiPAM, a silicon photonic accelerated memory pooling architecture designed to address the growing memory bandwidth and capacity demands of LLM workloads. SiPAM integrates DWDM-driven SiP I/Os directly along the entire shoreline of the compute die, replacing conventional HBMs and electrical interconnects with a unified high-bandwidth and energy-efficient photonic domain. This approach decouples memory bandwidth and capacity scaling from HBM packaging constraints by enabling high bandwidth access to a shared memory pool. Through analytical evaluations on Transformer-based workloads, we demonstrate that SiPAM is able to achieve up to 3.5× reduction in iteration time and up to 4.5× reduction in energy consumption when compared to state-of-the-art GPU systems. SiPAM also supports configurations otherwise infeasible on baseline systems by dynamically allocating memory capacity and bandwidth to match workload size and arithmetic intensity, enabling efficient parallelization across diverse workloads.

ACKNOWLEDGMENTS

This work was supported in part by the Laboratory for Physical Sciences (LPS) Research Initiative and the Center for Ubiquitous Connectivity (CUbiC), and is sponsored by Semiconductor Research Corporation (SRC) and Defense Advanced Research Projects Agency (DARPA) under the JUMP 2.0 program. The text in this submission was prepared with help from ChatGPT to improve grammar and clarity.

REFERENCES

1. A. Cho, A. Saxena, M. Qureshi, and A. Daglis, "COAXIAL: A CXL-centric memory system for scalable servers," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal. (SC)*, Piscataway, NJ, USA: IEEE Press, 2024, pp. 1–15, doi: 10.1109/SC41406.2024.00101.
2. Z. Wu and K. Bergman, "Silicon photonic accelerated memory pooling for efficient compute resource allocation," in *Proc. IEEE Symp. High-Perform. Interconnects (HOTI)*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 36–47, doi: 10.1109/HOTI66940.2025.00022.
3. M. Isaev et al., "Calculon: A methodology and tool for high-level co-design of systems and large language models," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, 2023, pp. 1–14.
4. Y. Wang et al., "Co-designed silicon photonics chip I/O for energy-efficient petascale connectivity," *IEEE Trans. Compon. Packag. Manuf. Technol.*, vol. 15, no. 8, pp. 1581–1591, Aug. 2025, doi: 10.1109/TCPMT.2024.3492189.
5. Z. Wu, B. Klenk, L. Dennison, and K. Bergman, "ACTINA: Adapting circuit-switching techniques for AI networking architectures," in *Proc. Int. Conf. High Perform. Comput., Netw., Storage Anal.*, Piscataway, NJ, USA: IEEE Press, 2025, pp. 1211–1222, doi: 10.1145/3712285.3759842.
6. Y. Ou, H. Zhang, A. Rovinski, D. Wentzlaff, and C. Batten, "Optically connected multi-stack HBM modules for large language model training and inference," *IEEE Comput. Archit. Lett.*, vol. 24, no. 1, Jan.–Jun. 2025, pp. 49–52, doi: 10.1109/LCA.2025.3540058.
7. "NVIDIA Blackwell datasheet (document ID: 3384703)," NVIDIA, Santa Clara, CA, USA, 2025. [Online]. Available: https://nvdam.widen.net/s/wwnsxrhm2w/blackwell-datasheet-3384703
8. "NVIDIA A100 tensor core GPU architecture," NVIDIA, Santa Clara, CA, USA, 2020. [Online]. Available: https://images.nvidia.com/aem-dam/en-zz/Solutions/data-center/nvidia-ampere-architecture-whitepaper.pdf
9. "NVIDIA H100 tensor core GPU architecture," NVIDIA, Santa Clara, CA, USA, 2022. [Online]. Available: https://resources.nvidia.com/en-us-data-center-overview/gtc22-whitepaper-hopper
10. "NVIDIA Blackwell architecture technical brief," NVIDIA, Santa Clara, CA, USA, 2024. [Online].

Available: https://resources.nvidia.com/en-us-blackwell-architecture
11. "HBM roadmap ver. 1.7 workshop presentation," KAIST TeraByte Interconnection and Package Lab., Daejeon, South Korea, Tech. Presentation, Jun. 2025. [Online]. Available: https://www.aionet.tech/KAIST_HBM_Roadmap.pdf
12. R. Merritt, "What is NVlink?" *NVIDIA*, Mar. 6, 2023. [Online]. Available: https://blogs.nvidia.com/blog/what-is-nvidia-nvlink/

ZHENGUO WU was with Columbia University, New York, NY, 10027, USA. He is now a research scientist at NVIDIA Corporation, Santa Clara, CA, 95051, USA. His research interests include optical interconnect and reconfigurable network design for machine learning systems. Wu received his Ph.D. degree in electrical engineering from Columbia University. Contact him at zw2542@columbia.edu.

YUYANG WANG was a postdoctoral research scientist with Columbia University, New York, NY, 10027, USA. He is now an assistant professor at the University of Connecticut, Storrs, CT, 06269, USA. His research interests include integrated photonics for communication and computing, electronic-photonic integration/packaging, and photonic design automation. Wang received his Ph.D. degree in electrical and computer engineering from the University of California, Santa Barbara. Contact him at yuyang.wang@uconn.edu.

KEREN BERGMAN is the Charles Batchelor Professor of Electrical Engineering at Columbia University. Her research interests include photonic interconnects and networks for computing. Bergman received her Ph.D. degree in electrical engineering from the Massachusetts Institute of Technology. She is a Fellow of IEEE and Optica. Contact her at bergman@ee.columbia.edu.

